

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication
Kind Code
Publication Date
Inventor(s)

20250279886
A1
September 04, 2025
Kao; Meng-Chao et al.

PROCESSING SYSTEM AND METHOD RELATED TO ENCRYPTION

Abstract

A processing system and method related to encryption are provided. The processing system includes multiple computing circuits. A memory stores data. The computing circuit performs a number theoretic transform (NTT) calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the NTT calculation. Therefore, the computing efficiency of encryption or decryption can be improved.

Inventors: Kao; Meng-Chao (New Taipei City, TW), Perng; Chiy Ferng (New Taipei City, TW), Shen; Tzu-Chiang (New Taipei City, TW), Shih; Cheng Jhih (Taipei, TW), Hung; Shih Hao (Taipei, TW)

Applicant: Wistron Corporation (New Taipei City, TW)

Family ID: 96141949

Assignee: Wistron Corporation (New Taipei City, TW)

Appl. No.: 18/657718

Filed: May 07, 2024

Foreign Application Priority Data

TW 113107218 Feb. 29, 2024

Publication Classification

Int. Cl.: H04L9/30 (20060101); G06F17/14 (20060101); G06F17/16 (20060101); H04L9/00 (20220101)

U.S. Cl.:

CPC H04L9/3093 (20130101); G06F17/142 (20130101); G06F17/16 (20130101); H04L9/008 (20130101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application claims the priority benefit of Taiwan application serial no. 113107218, filed on Feb. 29, 2024. The entirety of the above-mentioned patent application is hereby incorporated by reference herein and made a part of this specification.

BACKGROUND

Technical Field

[0002] The disclosure relates to a cryptographic technology, and in particular to a processing system and method related to encryption.

Description of Related Art

[0003] Today, data privacy and security issues have attracted much attention, especially when business data is stored in a hybrid multi-cloud environment and faces diverse security risks. However, there is an issue with traditional encryption methods, that is, data must be first decrypted before computation and analysis can be performed, which may lead to leakage of confidentiality during the process. In this case, the fully homomorphic encryption (FHE) technology has become a very promising solution.

[0004] Fully homomorphic encryption is a special encryption technology that allows calculations to be performed in an encrypted state and various operations to be executed on ciphertext without decrypting the ciphertext, which means that the calculations may be executed on encrypted data, and an obtained result is also encrypted. Only a person with a corresponding decryption key can decrypt the final result.

[0005] The main features of fully homomorphic encryption include: [0006] Additive homomorphism: allow an addition operation to be performed on encrypted numbers, and an obtained result is still in an encrypted form without the need for decryption. [0007] Multiplicative homomorphism: allow a multiplication computation to be performed on encrypted numbers, and an obtained result is still in an encrypted form also without the need for decryption. [0008] Fully homomorphic: support both additive and multiplicative homomorphism, allowing an arbitrarily complex calculation on encrypted data without the need for decryption.

[0009] Solutions to achieve fully homomorphic encryption include but are not limited to Rivest-Shamir-Adleman (RSA) homomorphic encryption, Gentry's fully homomorphic encryption over the integers, Brakerski-Gentry-Vaikuntanathan (BGV) fully homomorphic encryption scheme, etc. It is worth noting that the RSA only has "multiplicative" homomorphism, but does not have "additive" homomorphism, so the RSA is a partially homomorphic encryption (PHE).

[0010] Although fully homomorphic encryption provides strong privacy protection and security, computing costs are usually high, resulting in relatively slow encryption and decryption processes while requiring a large amount of computing resources. Therefore, in practical applications, fully

homomorphic encryption is still in the research and development stage and is being continuously improved to improve efficiency and effectiveness. [0011] The existing fully homomorphic encryption technology still has many limitations in practical applications. For example, one of the major issues is computing efficiency. The traditional fully homomorphic encryption technology requires repeated decryption and encryption of data, introducing additional time and computing costs during the computing process. The inefficiency of such a computing manner becomes particularly obvious in scenarios such as real-time data processing, cloud computing, and the Internet of Things.

SUMMARY

[0012] The disclosure provides a processing system and method related to encryption, which can improve the computing efficiency of a homomorphic computation, encryption, and decryption.

[0013] A processing system related to encryption of an embodiment of the disclosure includes (but is not limited to) multiple computing circuits. A memory stores data. The computing circuit performs a fast number theoretic transform (NTT) calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0014] A method related to encryption of an embodiment of the disclosure includes (but is not limited to) the following steps. A fast number theoretic transform calculation is performed on a polynomial through multiple computing circuits. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0015] In order for the features and advantages of the disclosure to be more comprehensible, the following specific embodiments are described in detail in conjunction with the drawings.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0016] FIG. 1 is an element block diagram of a processing system according to an embodiment of the disclosure.

[0017] FIG. 2 is a schematic diagram of a butterfly computation according to an embodiment of the disclosure.

[0018] FIG. 3 is a system architecture diagram of two-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0019] FIG. 4A is a schematic diagram illustrating a butterfly computation according to an embodiment of the disclosure.

[0020] FIG. 4B is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure.

[0021] FIG. 4C is a schematic diagram illustrating factor position swapping according to an embodiment of the disclosure.

[0022] FIG. 5A to FIG. 5C are schematic diagrams illustrating one-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0023] FIG. 6A to FIG. 6D are schematic diagrams illustrating two-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0024] FIG. 7 is a system architecture diagram of three-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0025] FIG. 8A is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure.

[0026] FIG. 8B is a schematic diagram illustrating memory address change according to an embodiment of the disclosure.

[0027] FIG. 9A to FIG. 9F are schematic diagrams illustrating three-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0028] FIG. 10 is a system architecture diagram of four-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0029] FIG. 11A to FIG. 11F are schematic diagrams illustrating four-dimensional fast number theoretic transform according to an embodiment of the disclosure.

[0030] FIG. 12 is a system architecture diagram of non-pipelined fast number theoretic transform according to an embodiment of the disclosure.

[0031] FIG. 13 is a schematic diagram illustrating data storage according to an embodiment of the disclosure.

[0032] FIG. 14A is a schematic diagram illustrating forward fast number theoretic transform according to an embodiment of the disclosure.

[0033] FIG. 14B is a schematic diagram illustrating non-pipelined fast number theoretic transform according to an embodiment of the disclosure.

[0034] FIG. 15 is a flowchart of an encryption method according to an embodiment of the disclosure.

DESCRIPTION OF THE EMBODIMENTS

[0035] FIG. 1 is a block diagram of a processing system 10 according to an embodiment of the disclosure. Please refer to FIG. 1. The processing system 10 includes (but is not limited to) one or more memories 11, a data conversion circuits 12, and multiple computing circuits 13.

[0036] The processing system 10 may be an electronic device such as a server, a desktop computer, a notebook computer, a smart phone, a tablet computer, a wearable device, a vehicle-mounted device, or a smart home appliance, or may be a central processing unit (CPU), a graphics processing unit (GPU), a programmable general-purpose or specific-purpose microprocessor, a digital signal processor (DSP), a programmable controller, a field programmable gate array (FPGA), an application-specific integrated circuit (ASIC), a computing accelerator, other similar elements, or a combination of the above elements.

[0037] The memory 11 may be any type of fixed or removable random access memory (RAM), read only memory (ROM), or similar elements. In an embodiment, the memory 11 is used to store program codes, software modules, configurations, data (for example, factors, algorithms, matrices, or parameters), or files, and an embodiment thereof will be described in detail later.

[0038] The data conversion circuit 12 is coupled to the memory 11. The data conversion circuit 12 may be various types of processors, controllers, processing elements (PE), processing cores, FPGA, ASIC, or other digital circuits.

[0039] The computing circuit 13 is coupled to the memory 11 and the data conversion circuit 12. The computing circuit 13 may be various types of processors, controllers, processing elements, processing cores, FPGA, ASIC, or other digital circuits.

[0040] In the following, the embodiments of the disclosure will be further explained below in conjunction with various devices, elements, and modules in the processing system 10. Each process of the embodiments of the disclosure may be adjusted according to the implementation situation.

[0041] The main concept of fully homomorphic encryption is to perform a computation on ciphertext to obtain an encrypted result, and a result obtained by decrypting the encrypted result is consistent with a result of performing the same computation on plaintext. It is worth noting that the basic computation of fully homomorphic encryption usually involves a multiplication computation of a polynomial. Fast number theoretic transform (NTT) is an algorithm that implements polynomial multiplication. A matrix multiplication calculation may be performed on a polynomial through a fast number theoretic transform calculation. Fast number theoretic transform is the fast Fourier transform (FFT) in integer form. Fast number theoretic transform may reduce the original time complexity of degree N polynomial multiplication from $O(N^2)$ to $O(N \log N)$. For example, fast number theoretic transform converts a polynomial computation into a point computation in a fast number theoretic transform domain, thereby greatly improving computing efficiency.

[0042] The mathematical expression of fast number theoretic transform may be:

[00001] $X_j = \sum_{i=0}^{N-1} \omega^{ij} x_i$ (1) [0043] where x_i is an i-th data vector, ω is a factor of a j-th degree N polynomial (may be presented by a matrix that records factors in the polynomial), and X_j is a j-th output matrix. Equation (1) above is matrix-vector multiplication, and the matrix used may be a Vandermonde matrix.

[0044] Since ω is periodic, Equation (1) above may be further simplified to only require multiplication of $O(N \log N)$ to complete all computations. For example, FIG. 2 is a schematic diagram of a butterfly computation according to an embodiment of the disclosure. Please refer to FIG. 2. Fast

number theoretic transform/fast Fourier transform can be implemented using a butterfly architecture. In the multiplication of two polynomials, $2x.\text{sup.}3+7x.\text{sup.}2+1x.\text{sup.}1+8x.\text{sup.}0$ and $2x.\text{sup.}3+0x.\text{sup.}2+4x.\text{sup.}1+8x.\text{sup.}0$, the polynomial $2x.\text{sup.}3+7x.\text{sup.}2+1x.\text{sup.}1+8x.\text{sup.}0$ is converted into a data vector [2, 7, 1, 8], and the polynomial $2x.\text{sup.}3+0x.\text{sup.}2+4x.\text{sup.}1+8x.\text{sup.}0$ is converted into a data vector [2, 0, 4, 8]. The two data vectors are first converted into the fast number theoretic transform domain via time complexity of $O(N\log N)$, and a point multiplication computation (that is, element-wise multiplication) is performed in the fast number theoretic transform domain. The point multiplication computation only takes time complexity of $O(N)$. Finally, time complexity of $O(N\log N)$ is taken to convert back to the normal domain. It should be noted that in the drawing, “x” is multiplication (multiply by the factor on the side, such as 2×4), “+” is addition (add two values on a connecting line, such as $(7\times 4)+8$), and “-” is subtraction (subtract two values on a connecting line, such as $(2\times 4)-1$).

[0045] In addition, in the drawing, the polynomial $x.\text{sup.}N+1$ is taken as an example, so FIG. 2 shows negacyclic fast number theoretic transform. The polynomial $x.\text{sup.}N+1$ is substituted into Equation (1) to become $X.\text{sub.}j=\sum.\text{sub.}j(\omega.\text{sup.}2j+1).\text{sup.}ix.\text{sub.}i$, so ω becomes a degree $2N$ polynomial and may be implemented using different conversion manners.

[0046] In an embodiment, in the computation of fast number theoretic transform, there are $\log N$ stages ($N\log N$ multiplications), and the hardware implementation includes full pipelined and non-full pipelined manners.

[0047] The full pipelined manner allocates the hardware (for example, the computing circuit 13, the memory 11, or a combination thereof) required by all stages, so more hardware resources are available for use, and there is better performance in output/cycle.

[0048] The non-pipelined manner may use the same set of hardware to implement different stages of operations, so there are fewer hardware resources available for use, but control is easier.

[0049] In an embodiment, the computing circuits 13 perform a fast number theoretic transform calculation on a polynomial. A matrix multiplication calculation is performed on the polynomial through the fast number theoretic transform calculation.

[0050] In an embodiment, the computing circuits 13 perform multi-dimensional fast number theoretic transform. The multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a computation architecture of a multi-stage matrix multiplication calculation through multiple sub-computations. One computing circuit 13 performs one sub-computation, and each sub-computation is located at one stage of the multi-stage computation architecture.

[0051] The mathematical expression of multi-dimensional fast number theoretic transform is:

$$[00002] X_{k_1, k_2, \dots, k_d} = \text{Math}_{n_1=0}^{N_1-1} \left(\text{Math}_{n_2=0}^{N_2-1} \dots \text{Math}_{n_d=0}^{N_d-1} X_{n_1, n_2, \dots, n_d} \right) \quad (2) \quad [0052] \text{ where}$$

$X.\text{sub.}k.\text{sub.}1.\text{sub.}, k.\text{sub.}2.\text{sub.}, \dots, k.\text{sub.}d.\text{sub.}$ is the output factor of $(k.\text{sub.}1, k.\text{sub.}2, \dots, k.\text{sub.}d)$ -th multi-dimensional fast number theoretic transform, $x.\text{sub.}n.\text{sub.}1.\text{sub.}, n.\text{sub.}2.\text{sub.}, \dots, n.\text{sub.}d.\text{sub.}$ is the $(n.\text{sub.}1, n.\text{sub.}2, \dots, n.\text{sub.}d)$ -th input matrix, ω is a twiddle factor, $N.\text{sub.}1*N.\text{sub.}2*\dots*N.\text{sub.}d$ is N , d is a target dimension, and N is the degree of the polynomial.

[0053] FIG. 3 is a system architecture diagram of two-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 3, which shows a two-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a two-dimensional computation (d of Equation (2) is 2), the mathematical expression of multi-dimensional fast number theoretic transform is:

$$[00003] X_{N_2 k_1 + k_2} = \text{Math}_{n_1=0}^{N_1-1} \text{Math}_{n_2=0}^{N_2-1} X_{N_1 n_2 + n_1} \left(\omega^{2(N_2 k_1 + k_2) + 1} \right)^{N_1 n_2 + n_1} = \text{Math}_{n_1=0}^{N_1-1} \left(\left(\text{Math}_{n_2=0}^{N_2-1} X_{N_1 n_2 + n_1} \left(\omega^{2k_2 + 1} \right)^{N_1 n_2} \right)^{n_1(2k_2 + 1 - N_2)} \right)^{2k_1 + 1}$$

[0054] where $X.\text{sub.}N.\text{sub.}2.\text{sub.}k.\text{sub.}1.\text{sub.}+k.\text{sub.}2$ is the output factor of $(N.\text{sub.}2k.\text{sub.}1+k.\text{sub.}2)$ -th multi-dimensional fast number theoretic transform, $x.\text{sub.}N.\text{sub.}1.\text{sub.}n.\text{sub.}2.\text{sub.}+n.\text{sub.}1$ is the $(N.\text{sub.}1n.\text{sub.}2+n.\text{sub.}1)$ -th input matrix, ω is the twiddle factor, $N.\text{sub.}1*N.\text{sub.}2$ is N , and N is the degree of the polynomial.

[0055] Equation (3) may be allocated to the computing circuits 13 for implementation. The computing circuits 13 include one or more stage conversion circuits, one or more first twiddle circuits, and one or more transposition circuits. Taking FIG. 3 as an example, the computing circuits 13 with such an architecture include two stage conversion circuits 13.sub.1-1 and 13.sub.1-4, a first twiddle circuit 13.sub.1-2, and a transposition circuit 13.sub.1-3. The order of execution of the computing circuits 13 is the stage conversion circuit 13.sub.1-1, the first twiddle circuit 13.sub.1-2, the transposition circuit 13.sub.1-3, and the stage conversion circuit 13.sub.1-4.

[0056] In an embodiment, the sub-computations of the stage conversion circuits 13.sub.1-1 and 13.sub.1-4 are butterfly computations. FIG. 4A is a schematic diagram illustrating a butterfly computation according to an embodiment of the disclosure. Please refer to FIG. 4A. A sub-stage of the butterfly computation is related to the degree (N) of the polynomial, that is, $\log N$. In an m -th sub-stage, the butterfly computation of

$$[00004] \sqrt[N]{N} / (2^{\text{Math.} m})$$

points is performed, where d is the target dimension (two-dimensional as shown in FIG. 3, that is, d is 2). For example, in a first sub-stage, one of the outputs is $x.\text{sub.}1-1-00+x.\text{sub.}1-1-(i+1).\text{Math.}\omega 0$ (each multiplication “x” has a corresponding twiddle factor ω), where $i+1$ is $M/2+0$. A data vector $[x.\text{sub.}1-1-00, \dots, x.\text{sub.}1-1-M]$ has M values, and M is $\sqrt[N]{N}$ {square root over (N)}. By analogy, another output is $x.\text{sub.}1-1-01+x.\text{sub.}1-1-(i+2).\text{Math.}\omega 1$. The next output is $x.\text{sub.}1-1-i1+x.\text{sub.}1-1-M.\text{Math.}\omega 2$. Finally, the output factors of a $\log N$ -th sub-stage are $X.\text{sub.}1-2-00, X.\text{sub.}1-2-01, X.\text{sub.}1-2-02, X.\text{sub.}1-2-03, \dots, X.\text{sub.}1-2-(M-1), X.\text{sub.}1-2-M$.

[0057] In an embodiment, the sub-computation of the first twiddle circuit 13.sub.1-2 is a multiplication computation of the twiddle factor. FIG. 4B is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure. Please refer to FIG. 4B. A multiplication factor is multiplied by the twiddle factor, and a multiplication result is used as a starting twiddle factor. For example, the starting twiddle factor is $\omega \{ \text{circumflex over () } (n(1-N.\text{sub.}2)) \}$, and the multiplication factor is $n \{ \text{circumflex over () } (2n) \}$, where n is $0 \sim N1-1$. In addition, the multiplication result is further multiplied by the input factor.

[0058] In an embodiment, the sub-computation of the transposition circuit 13.sub.1-3 is factor position swapping. FIG. 4C is a schematic diagram illustrating factor position swapping according to an embodiment of the disclosure. Please refer to FIG. 4C. The sub-computation performs transposition on a matrix

$$[00005] \begin{bmatrix} A & B \\ C & D \end{bmatrix},$$

that is,

$$[00006] \begin{bmatrix} A & B \\ C & D \end{bmatrix}^T = \begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}.$$

First, the positions of the matrices A and C are swapped, so that the matrix C is stored in a temporary storage area/register of the memory 11, and the matrices A and D are stored in another temporary storage area/register of the memory 11. If the matrix C is stored, storing of the matrix B will be skipped. If the matrices A and B are input at the same time, the positions of the matrices A and B will be swapped. Then, positions are swapped again. For example, if the matrices A and B are not input, the positions will not be swapped.

[0059] Please refer to FIG. 3. The data conversion circuit 12 obtains the input matrix. The column number or the row number of the input matrix is $\sqrt[N]{N}$ {square root over (N)}, such as $\sqrt[N]{N}$. Multiple elements of the input matrix are factors of the polynomial used for encryption, such as the factors of the polynomial related to ciphertext or plaintext used for fully homomorphic encryption. In an embodiment, the elements of the input matrix are respectively the same as multiple elements in input data (one-to-one element correspondence). The input data is also the factors of the same polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext.

[0060] In an embodiment, the data conversion circuit 12 converts the input data into the input matrix according to the target dimension. The target dimension is greater than one dimension (two dimensions in the embodiment). Therefore, the data conversion circuit 12 vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The elements in the input matrix are the elements in the input data in vector form (values thereof are the factors of the polynomial). In other words, “converting” a vector into a matrix is to allocate the elements in the input data to corresponding positions in the input matrix. For example, a first element of the input data is allocated to a first row and a first column of the input matrix.

[0061] In response to the target dimension being two-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (3). Next, the butterfly computation is performed on the input matrix through the stage conversion circuit 13.sub.1-1 of $\log \sqrt{\text{square root over (N)}}$ sub-stages, the multiplication computation of the twiddle factor is performed on the output factor of the butterfly computation through the first twiddle circuit 13.sub.1-2, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit 13.sub.1-3, and the butterfly computation is performed on the output factor of position transposition through the stage conversion circuit 13.sub.1-4 of $\log \sqrt{\text{square root over (N)}}$ sub-stages. That is, two-dimensional multi-dimensional fast number theoretic transform is performed on the input matrix through the computing circuits 13 (that is, the two stage conversion circuits 13.sub.1-1 and 13.sub.1-4, the first twiddle circuit 13.sub.1-2, and the transposition circuit 13.sub.1-3).

[0062] FIG. 5A to FIG. 5C are schematic diagrams illustrating one-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 5A to FIG. 5C. For example, the input data is [00, 01, 02, 03, 14, 05, 06, 07, 08, 09, 10, 11, 12]. It should be noted that in the drawings, “ $\times$ ” is multiplication (multiply by the twiddle factor on the side, such as 15×22), “+” is addition (add two values on a connecting line, such as $(15 \times 22) + 08$), and “−” is subtraction (subtract two values on a connecting line, such as $(15 \times 22) - 08$). The twiddle factor of a first stage is $(-1)^{\text{circumflex over ()}(\frac{1}{2})}$ (for example, $22 \pmod{97}$), and the twiddle factors of a second stage are $(-1)^{\text{circumflex over ()}(\frac{1}{4})}$ (for example, $64 \pmod{97}$) and $(-1)^{\text{circumflex over ()}(\frac{3}{4})}$ (for example, $50 \pmod{97}$), and so on. Finally, the output of the input data after fast number theoretic transform is [96, 89, 34, 87, 67, 73, 10, 74, 73, 09, 78, 88, 41, 36, 75].

[0063] FIG. 6A to FIG. 6D are schematic diagrams illustrating two-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 6A to FIG. 6D. The degree of the polynomial is $N = N_{\text{sub.1}} \cdot N_{\text{sub.2}}$ (for example, $N_{\text{sub.1}} = N_{\text{sub.2}} = \sqrt{\text{square root over (N)}}$). An index of the polynomial is represented by $x_{\text{sub.N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$. An inner layer $n_{\text{sub.2}}$ is added up to fast number theoretic transform of the first stage. $\omega_{\text{sup.n.sub.1.sub.}(2k_{\text{sup.2.sub.+1-N.sub.2.sub.)}}$ is the twiddle factor used in the multiplication computation. An outer layer $n_{\text{sub.1}}$ is added up to fast number theoretic transform of the second stage. The hardware architecture of FIG. 3 is suitable for implementing both forward and inverse fast number theoretic transform, and the difference is that the remaining hardware pipelined part may be shared by adding one or two additional stages of specific multiplication computations.

[0064] The data conversion circuit converts 1×16 input data into a 4×4 input matrix, such as

[00007]
$$\begin{bmatrix} 15 & 11 & 07 & 03 \\ 14 & 10 & 06 & 02 \\ 13 & 09 & 05 & 01 \\ 12 & 08 & 04 & 00 \end{bmatrix}.$$

[0065] In Step S601, the multiplication computation is performed on the twiddle factor $\omega_{\text{sup.2k.sub.2.sub.+1.sub.N.sub.1.sub.n.sub.2}}$. In Step S602, the multiplication computation is performed on the twiddle factor $\omega_{\text{sup.n.sub.1.sub.}(2k_{\text{sup.2.sub.+1-N.sub.2.sub.)}}$. In Step S603, the multiplication computation is performed on the twiddle factor $\omega_{\text{sup.2k.sub.2.sub.+1.sub.N.sub.1.sub.n.sub.2}}$. The “twiddle” processing in the drawings is, for example, the sub-computation shown in FIG. 4B, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the “transposition” processing in the drawings is, for example, the sub-computation shown in FIG. 4C, that is, factor position swapping. Finally, the output matrix is

[00008]
$$\begin{bmatrix} 75 & 78 & 10 & 34 \\ 36 & 09 & 73 & 03 \\ 41 & 73 & 67 & 89 \\ 88 & 74 & 87 & 96 \end{bmatrix},$$

and the values are the same as the values in the output vector of FIG. 5C (the difference is only in the positions of the matrix).

[0066] The twiddle factors in Equation (3) are $\omega_{\text{sup.2k+1}} \{k=0 \sim N_{\text{sub.2}}-1\}$ and $\omega_{\text{sup.n.sub.1.sub.}(2k_{\text{sup.2.sub.+1-N.sub.2.sub.)}}$. In an embodiment, the twiddle factors may be prestored in the memory 11 for used in subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the twiddle factors $\omega_{\text{sup.n}(1-N_{\text{sub.2.sub.)}}$ and $\omega_{\text{sup.2n}\{n=0 \sim N_{\text{sub.1}}-1\}}$ may be computed on the fly to generate the twiddle factor $\omega_{\text{sup.n.sub.1.sub.}(2k_{\text{sup.2.sub.+1-N.sub.2.sub.)}}$. At this time, the amount of data required to be stored is $3\sqrt{\text{square root over (N)}}$.

[0067] FIG. 7 is a system architecture diagram of three-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 7, which shows a three-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a three-dimensional computation (d of Equation (2) is 3), the mathematical expression of multi-dimensional fast number theoretic transform is:

[00009]

$$X_{N_3 N_2 k_1 + N_3 k_2 + k_3} = \text{Math}_{n_1=0}^{N_1-1} \cdot \text{Math}_{n_2=0}^{N_2-1} \cdot \text{Math}_{n_3=0}^{N_3-1} X_{N_1 N_2 n_3 + N_1 n_2 + n_1} \left(\omega_{N_3 N_2 k_1 + N_3 k_2 + k_3}^{2(N_3 N_2 k_1 + N_3 k_2 + k_3) + 1} \right)^{N_1 N_2 n_3 + N_1 n_2 + n_1} = \text{Math}_{n_1=0}^{N_1-1} \left(\text{Math}_{n_2=0}^{N_2-1} \left(\right. \right.$$

[0068] where $X_{\text{sub.N.sub.3.sub.2.sub.k.sub.1.sub.+N.sub.3.sub.k.sub.2.sub.+k.sub.3}}$ is the output factor of

$(N_{\text{sub.3}} \cdot N_{\text{sub.2k.sub.1+N.sub.3k.sub.2+k.sub.3}})$ -th multi-dimensional fast number theoretic transform,

$x_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$ is the $(N_{\text{sub.1}} \cdot N_{\text{sub.2n.sub.3+N.sub.1n.sub.2+n.sub.1}})$ -th input matrix, ω is the twiddle factor, $N_{\text{sub.1}} \cdot N_{\text{sub.2}} \cdot N_{\text{sub.3}}$ is N and N is the degree of the polynomial.

[0069] Equation (4) may be allocated to the computing circuits 13 for implementation. The computing circuits 13 include one or more first twiddle circuits, one or more second twiddle circuits, one or more stage conversion circuits, one or more transposition circuits, and one or more memory transposition circuits. Taking FIG. 7 as an example, the computing circuits 13 with such an architecture include two first twiddle circuits 13.sub.2-1 and 13.sub.2-13, two second twiddle circuits 13.sub.2-2 and 13.sub.2-12, three stage conversion circuits 13.sub.2-3, 13.sub.2-7, and 13.sub.2-11, four transposition circuits 13.sub.2-4, 13.sub.2-6, 13.sub.2-8, and 13.sub.2-10, and two memory transposition circuits 13.sub.2-5 and 13.sub.2-9. The order of execution of the computing circuits 13 is the first twiddle circuit 13.sub.2-1, the second twiddle circuit 13.sub.2-2, the stage conversion circuit 13.sub.2-3, the transposition circuit 13.sub.2-4, the memory transposition circuit 13.sub.2-5, the transposition circuit 13.sub.2-6, the stage conversion circuit 13.sub.2-7, the transposition circuit 13.sub.2-8, the memory transposition circuit 13.sub.2-9, the transposition circuit 13.sub.2-10, the stage conversion circuit 13.sub.2-11, the second twiddle circuit 13.sub.2-12, and the first twiddle circuit 13.sub.2-13.

[0070] In an embodiment, the sub-computations of the first twiddle circuits 13.sub.2-1 and 13.sub.2-13 are the multiplication computations of the twiddle factors, and reference may be made to the description of FIG. 4B.

[0071] In an embodiment, the sub-computations of the second twiddle circuits 13.sub.2-2 and 13.sub.2-12 are the multiplication computation of the twiddle factors. FIG. 8A is a schematic diagram illustrating a multiplication computation with a twiddle factor according to an embodiment of the disclosure. Please refer to FIG. 8A. The output of the previous stage is used as the input, and the input is multiplied by the twiddle factor to obtain

the output.

[0072] In an embodiment, the sub-computations of the stage conversion circuits **13.sub.2-3**, **13.sub.2-7**, and **13.sub.2-11** are the butterfly computations, and reference may be made to the description of FIG. **4A**.

[0073] In an embodiment, the sub-computations of the transposition circuits **13.sub.2-4**, **13.sub.2-6**, **13.sub.2-8**, and **13.sub.2-10** are factor position swapping, and reference may be made to the description of FIG. **4C**.

[0074] In an embodiment, the sub-computations of the memory transposition circuits **13.sub.2-5** and **13.sub.2-9** are memory address change. FIG. **8B** is a schematic diagram illustrating memory address change according to an embodiment of the disclosure. Please refer to FIG. **8B**. Two 4×4 matrices (for example

[00010]
$$\begin{bmatrix} 31 & 30 & 29 & 28 \\ 27 & 26 & 25 & 24 \\ 23 & 22 & 21 & 20 \\ 19 & 18 & 17 & 16 \end{bmatrix} \text{ and } \begin{bmatrix} 15 & 14 & 13 & 12 \\ 11 & 10 & 09 & 08 \\ 07 & 06 & 05 & 04 \\ 03 & 02 & 01 & 00 \end{bmatrix}$$

on the left side of the drawing are extended and combined into a 8×4 matrix on the right side of the drawing. Next, a write address of each element in the 8×4 matrix in the memory **11** is changed. For example, each column in the matrix is converted to binary form. Taking the 8×4 matrix as an example, elements in each column are converted into three bits: abc are converted into bca (001->010).

[0075] Please refer to FIG. **7**. The data conversion circuit **12** converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (three dimensions in the embodiment). Therefore, the data conversion circuit **12** vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The column number or the row number of the input matrix is

[00011] $\sqrt[N]{N}$,

such as

[00012] $\sqrt[N]{N}$.

[0076] In response to the target dimension being three-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (4). Then, the multiplication computation is performed on the input matrix and the corresponding twiddle factor through the first twiddle circuit **13.sub.2-1**, the multiplication computation is performed on the output factor of the multiplication computation and the corresponding twiddle factor through the second twiddle circuit **13.sub.2-2**, the butterfly computation is performed on the output factor of the multiplication computation through the stage conversion circuit **13.sub.2-3** of $\log \sqrt[N]{N}$ sub-stages, factor position swapping is performed on the output factor of the butterfly computation through the transposition circuit **13.sub.2-4**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13.sub.2-5**, factor position swapping is performed on the address-changed factor through the transposition circuit **13.sub.2-6**, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit **13.sub.2-7** of $\log \sqrt[N]{N}$ sub-stages, factor position swapping is performed on the output factor of the butterfly computation through the transposition circuit **13.sub.2-8**, memory address change is performed on the position-swapped factor through the memory transposition circuit **13.sub.2-9**, factor position swapping is performed on the address-changed factor through the transposition circuit **13.sub.2-10**, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit **13.sub.2-11** of $\log \sqrt[N]{N}$ sub-stages (d is 3), the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the second twiddle circuit **13.sub.2-12**, and the multiplication computation is performed on the output factor of the multiplication computation and the corresponding twiddle factor through the first twiddle circuit **13.sub.2-13**. That is, three-dimensional multi-dimensional fast number theoretic transform is performed on the input matrix through the computing circuits **13** (that is, the two first twiddle circuits **13.sub.2-1** and **13.sub.2-13**, the two second twiddle circuits **13.sub.2-2** and **13.sub.2-12**, the four transposition circuits **13.sub.2-4**, **13.sub.2-6**, **13.sub.2-8**, and **13.sub.2-10**, and the two memory transposition circuits **13.sub.2-5** and **13.sub.2-9**).

[0077] FIG. **9A** to FIG. **9F** are schematic diagrams illustrating three-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **9A** to FIG. **9F**. $\omega_{N^2}^{sup.N.sup.1.sup.N.sup.2.sup.n.sup.3}$, $(\omega_{2k}^{sup.3})_{sup.N.sup.1.sup.N.sup.2.sup.n.sup.3}$, $(\omega_{2k}^{sup.3.sup.+1})_{sup.N.sup.1.sup.n.sup.2.sup.+n.sup.1}$, $(\omega_{2k}^{sup.2})_{sup.N.sup.3.sup.N.sup.1.sup.n.sup.2}$, $(\omega_{2N}^{sup.3.sup.k.sup.2})_{sup.N.sup.1.sup.n.sup.2.sup.+n.sup.1}$, and $(\omega_{2k}^{sup.1})_{sup.N.sup.3.sup.N.sup.2.sup.n.sup.1}$ are the twiddle factors used in the multiplication computations. The data conversion circuit converts 1×16 input data into two 4×2 input matrices, such as

[00013]
$$\begin{bmatrix} 11 & 03 \\ 10 & 02 \\ 09 & 01 \\ 08 & 00 \end{bmatrix} \text{ and } \begin{bmatrix} 15 & 07 \\ 14 & 06 \\ 13 & 05 \\ 12 & 04 \end{bmatrix}.$$

That is, one dimension is added to store or compute another two-dimensional matrix. In Step **S901**, the multiplication computation is performed on the twiddle factor $\omega_{N^2}^{sup.N.sup.2.sup.N.sup.2.sup.n.sup.3}$ is multiplied, such as

[00014]
$$\begin{bmatrix} 11 & 03 \\ 10 & 02 \\ 09 & 01 \\ 08 & 00 \end{bmatrix} \times \begin{bmatrix} 22 \\ 1 \end{bmatrix}.$$

In Step **S902**, the multiplication computation is performed on the twiddle factor $(\omega_{2k}^{sup.3})_{sup.N.sup.1.sup.N.sup.2.sup.n.sup.3}$. In Step **S903**, the multiplication computation is performed on the twiddle factor $(\omega_{2k}^{sup.3.sup.+1})_{sup.N.sup.1.sup.n.sup.2.sup.+n.sup.1}$. In Step **S904**, the multiplication computation is performed on the twiddle factor $(\omega_{2k}^{sup.2})_{sup.N.sup.3.sup.N.sup.1.sup.n.sup.2}$. In Step **S905**, the multiplication computation is performed on the twiddle factor $(\omega_{2N}^{sup.3.sup.k.sup.2})_{sup.N.sup.1.sup.n.sup.2.sup.+n.sup.1}$. In Step **S906**, the multiplication computation is performed on the twiddle factor $(\omega_{2k}^{sup.1})_{sup.N.sup.3.sup.N.sup.2.sup.n.sup.1}$. The “twiddle” processing in the drawings is, for example, the sub-computation shown in FIG. **4B**, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the “transposition” processing in the drawings is, for example, the sub-computation shown in FIG. **4C**, that is, factor position swapping. Finally, the output matrices

[00015]
$$\begin{bmatrix} 78 & 34 \\ 09 & 03 \\ 73 & 89 \\ 74 & 96 \end{bmatrix} \text{ and } \begin{bmatrix} 75 & 10 \\ 36 & 73 \\ 41 & 67 \\ 88 & 87 \end{bmatrix},$$

and the values are the same as the values in the output vector of FIG. **5C** and the output matrix of FIG. **6D** (the difference is only in the positions of the matrix).

[0078] In an embodiment, the twiddle factors may be prestored in the memory **11** for subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the generated twiddle factors may be computed on the fly using the multiplication factor and the starting twiddle factor. At this time, the amount of data required to be stored is $O(n \circ ())^{(1/2)}$.

[0079] FIG. **10** is a system architecture diagram of four-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. **10**, which shows a four-dimensional pipelined multi-dimensional fast number theoretic transform architecture. For a four-

dimensional computation (d of Equation (2) is 4), the mathematical expression of multi-dimensional fast number theoretic transform is:
[00016]

$$X_{N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4} = \text{Math}_{n_1=0}^{N_1-1} \cdot \text{Math}_{n_2=0}^{N_2-1} \cdot \text{Math}_{n_3=0}^{N_3-1} \cdot \text{Math}_{n_4=0}^{N_4-1} x_{N_1 N_2 N_3 n_4 + N_1 N_2 n_3 + N_1 n_2 + n_1} \left(2(N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4) + 1 \right)^{N_1 N_2 N_3}$$

[0080] where $X_{\text{sub.N.sub.4.sub.4.sub.N.sub.3.sub.2.sub.k.sub.1.sub.+N.sub.4.sub.N.sub.3.sub.k.sub.2.sub.+N.sub.4.sub.k.sub.3.sub.+k.sub.4}}$ is the output factor of $(N_{\text{sub.4}}N_{\text{sub.3}}N_{\text{sub.2}}k_{\text{sub.1}}+N_{\text{sub.4}}N_{\text{sub.3}}k_{\text{sub.2}}+N_{\text{sub.4}}k_{\text{sub.3}}+k_{\text{sub.4}})$ -th multi-dimensional fast number theoretic transform, $x_{\text{sub.N.sub.4.sub.4.sub.N.sub.3.sub.2.sub.k.sub.1.sub.+N.sub.4.sub.N.sub.3.sub.k.sub.2.sub.+N.sub.4.sub.k.sub.3.sub.+k.sub.4}}$ is the $(N_{\text{sub.4}}N_{\text{sub.3}}N_{\text{sub.2}}k_{\text{sub.1}}+N_{\text{sub.4}}N_{\text{sub.3}}k_{\text{sub.2}}+N_{\text{sub.4}}k_{\text{sub.3}}+k_{\text{sub.4}})$ -th input matrix, ω is the twiddle factor, $N_{\text{sub.1}}\text{Math}_{\text{sub.N.sub.2}}\text{Math}_{\text{sub.N.sub.3}}\text{Math}_{\text{sub.N.sub.4}}$ is $N_{\text{sub.4}}$, and N is the degree of the polynomial.

[0081] Equation (5) may be allocated to the computing circuits 13 for implementation. The computing circuits 13 include one or more first twiddle circuits, one or more stage conversion circuits, one or more transposition circuits, and one or more memory transposition circuits. Taking FIG. 10 as an example, the computing circuits 13 with such an architecture include four first twiddle circuits 13.sub.3-1, 13.sub.3-3, 13.sub.3-8, and 13.sub.3-13, four stage conversion circuits 13.sub.3-2, 13.sub.3-7, 13.sub.3-12, and 13.sub.3-16, five transposition circuits 13.sub.3-4, 13.sub.3-6, 13.sub.3-9, 13.sub.3-11, and 13.sub.3-14, and three memory transposition circuits 13.sub.3-5, 13.sub.3-10, and 13.sub.3-15. The order of execution of the computing circuits 13 is the first twiddle circuit 13.sub.3-1, the stage conversion circuit 13.sub.3-2, the first twiddle circuit 13.sub.3-3, the transposition circuit 13.sub.3-4, the memory transposition circuit 13.sub.3-5, the transposition circuit 13.sub.3-6, the stage conversion circuit 13.sub.3-7, the first twiddle circuit 13.sub.3-8, the transposition circuit 13.sub.3-9, the memory transposition circuit 13.sub.3-10, the transposition circuit 13.sub.3-11, the stage conversion circuit 13.sub.3-12, the first twiddle circuit 13.sub.3-13, the transposition circuit 13.sub.3-14, the memory transposition circuit 13.sub.3-15, and the stage conversion circuit 13.sub.3-16.

[0082] In an embodiment, the sub-computations of the first twiddle circuits 13.sub.3-1, 13.sub.3-3, 13.sub.3-8, and 13.sub.3-13 are the multiplication computations of the twiddle factors, and reference may be made to the description of FIG. 4B.

[0083] In an embodiment, the sub-computations of the stage conversion circuits 13.sub.3-2, 13.sub.3-7, 13.sub.3-12, and 13.sub.3-16 are the butterfly computations, and reference may be made to the description of FIG. 4A.

[0084] In an embodiment, the sub-computations of the transposition circuits 13.sub.3-4, 13.sub.3-6, 13.sub.3-9, 13.sub.3-11, and 13.sub.3-14 are factor position swapping, and reference may be made to the description of FIG. 4C.

[0085] In an embodiment, the sub-computations of the memory transposition circuits 13.sub.3-5, 13.sub.3-10, and 13.sub.3-15 are memory address change, and reference may be made to the description of FIG. 8B.

[0086] Please refer to FIG. 10. The data conversion circuit 12 converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (four dimensions in the embodiment). Therefore, the data conversion circuit 12 vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. The column number or the row number of the input matrix is $\sqrt[N]{d}$, such as $\sqrt[N]{d}$.

[0087] In response to the target dimension being four-dimensional, the mathematical expression of multi-dimensional fast number theoretic transform is determined as Equation (5). Then, the multiplication computation is performed on the input matrix and the corresponding twiddle factor through the first twiddle circuit 13.sub.3-1, the butterfly computation is performed on the output factor of the multiplication computation through the stage conversion circuit 13.sub.3-2 of $\log \sqrt[N]{d}$ sub-stages (d is 4), the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the first twiddle circuit 13.sub.3-3, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit 13.sub.3-4, memory address change is performed on the position-swapped factor through the memory transposition circuit 13.sub.3-5, memory address change is performed on the position-swapped factor through the transposition circuit 13.sub.3-6, the butterfly computation is performed on the address-changed factor through the stage conversion circuit 13.sub.3-7 of $\log \sqrt[4]{d}$ sub-stages, the multiplication computation with the corresponding twiddle factor is performed through the first twiddle circuit 13.sub.3-8, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit 13.sub.3-9, memory address change is performed on the position-swapped factor through the memory transposition circuit 13.sub.3-10, factor position swapping is performed on the address-changed factor through the transposition circuit 13.sub.3-11, the butterfly computation is performed on the position-swapped factor through the stage conversion circuit 13.sub.3-12 of $\log \sqrt[4]{d}$ sub-stages, the multiplication computation is performed on the output factor of the butterfly computation and the corresponding twiddle factor through the first twiddle circuit 13.sub.3-13, factor position swapping is performed on the output factor of the multiplication computation through the transposition circuit 13.sub.3-14, memory address change is performed on the position-swapped factor through the memory transposition circuit 13.sub.3-15, and the butterfly computation is performed on the address-changed factor through the stage conversion circuit 13.sub.3-16 of $\log \sqrt[4]{d}$ sub-stages.

[0088] FIG. 11A to FIG. 11F are schematic diagrams illustrating four-dimensional fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 11A to FIG. 11F. $\omega_{\text{sub.N.sub.1.sub.N.sub.2.sub.N.sub.3.sub.n.sub.4}}$,

$(\omega_{\text{sub.2k.sub.4}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.N.sub.3.sub.n.sub.4}}$,

$(\omega_{\text{sub.2k.sub.4.sub.+1}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$,

$(\omega_{\text{sub.2k.sub.3}})_{\text{sub.N.sub.4.sub.N.sub.2.sub.N.sub.1.sub.n.sub.3}}$,

$(\omega_{\text{sub.2N.sub.4.sub.k.sub.3}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$,

$(\omega_{\text{sub.2k.sub.2}})_{\text{sub.N.sub.4.sub.N.sub.3.sub.N.sub.1.sub.n.sub.3}}$, $(\omega_{\text{sub.2N.sub.4.sub.N.sub.3.sub.k.sub.2}})_{\text{sub.N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$,

and $(\omega_{\text{sub.2k.sub.1}})_{\text{sub.N.sub.4.sub.N.sub.3.sub.N.sub.2.sub.n.sub.1}}$ are the twiddle factors used in the multiplication computations. The data conversion circuit converts 1×16 input data into four 2×2 input matrices, such as

$$[00017] \begin{bmatrix} 12 & 04 \\ 08 & 00 \end{bmatrix}, \begin{bmatrix} 13 & 05 \\ 09 & 01 \end{bmatrix}, \begin{bmatrix} 14 & 06 \\ 10 & 02 \end{bmatrix}, \text{ and } \begin{bmatrix} 15 & 07 \\ 11 & 03 \end{bmatrix}.$$

That is, two dimensions are added to store or compute another two two-dimensional matrices. In Step S1101, the multiplication computation is performed on the twiddle factor $\omega_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$, such as

$$[00018] \begin{bmatrix} 12 & 04 \\ 08 & 00 \end{bmatrix} \times \begin{bmatrix} 22 \\ 1 \end{bmatrix}.$$

In Step S1102, the multiplication computation is performed on the twiddle factor $(\omega_{\text{sub.2k.sub.4}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.N.sub.3.sub.n.sub.4}}$. In Step S1103, the multiplication computation is performed on the twiddle factor

$(\omega_{\text{sub.2k.sub.4.sub.+1}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$. In Step S1104, the multiplication computation is

performed on the twiddle factor $(\omega_{\text{sub.2k.sub.3}})_{\text{sub.N.sub.4.sub.N.sub.2.sub.N.sub.1.sub.n.sub.3}}$. In Step S1105, the multiplication computation is

performed on the twiddle factor $(\omega_{\text{sub.2N.sub.4.sub.k.sub.3}})_{\text{sub.N.sub.1.sub.N.sub.2.sub.n.sub.3.sub.+N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$. In Step

S1106, the multiplication computation is performed on the twiddle factor $(\omega_{\text{sub.2k.sub.2}})_{\text{sub.N.sub.4.sub.N.sub.3.sub.N.sub.1.sub.n.sub.3}}$. In Step

S1107, the multiplication computation is performed on the twiddle factor

$(\omega_{\text{sub.2N.sub.4.sub.N.sub.3.sub.k.sub.2}})_{\text{sub.N.sub.1.sub.n.sub.2.sub.+n.sub.1}}$. In Step S1108, the multiplication computation is performed on the

twiddle factor (ω .sup.2k.sup.1).sup.N.sup.1).sup.N.sup.3.sup.N.sup.2.sup.N.sup.1). The “twiddle” processing in the drawings is, for example, the sub-computation shown in FIG. 4B, that is, the multiplication computation is performed on the corresponding twiddle factor. In addition, the “transposition” processing in the drawings is, for example, the sub-computation shown in FIG. 4C, that is, factor position swapping. Finally, the output matrix is

[00019][

88	87
74	96

], [

41	67
73	89

], [

36	73
09	03

], and [

75	10
78	34

],

and the values are the same as the values in the output vector of FIG. 5C and the output matrices of FIG. 6D and FIG. 9F (the difference is only in the positions of the matrix).

[0089] In an embodiment, the twiddle factors may be prestored in the memory 11 for subsequent computations. In another embodiment, some or all of the twiddle factors may be generated on the fly or dynamically. For example, the resulting twiddle factor may be computed on the fly using the multiplication factor and the starting twiddle factor. At this time, the amount of data required to be stored is $O(n \circlearrowleft (\frac{1}{2}))$.

[0090] FIG. 12 is a system architecture diagram of non-pipelined fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 12, which shows a non-pipelined fast number theoretic transform architecture. Such an architecture may be implemented through the computing circuits 13. The computing circuits 13 include one or more stage conversion circuits and one or more memory circuits. Taking FIG. 12 as an example, the computing circuits 13 with such an architecture include a stage conversion circuit 13.sub.4-1 and a memory circuit 13.sub.4-2. The order of execution of the computing circuits 13 is the stage conversion circuit 13.sub.4-1 and the memory circuit 13.sub.4-2.

[0091] In an embodiment, the sub-computation of the stage conversion circuit 13.sub.4-1 is the butterfly computation, and reference may be made to the description of FIG. 4A.

[0092] In an embodiment, the sub-computation of the memory circuit 13.sub.4-2 is to store data to one or more memories 11. FIG. 13 is a schematic diagram illustrating data storage according to an embodiment of the disclosure. Please refer to FIG. 13. Data is accessed from the memory 11, the butterfly computation is performed on the accessed data, and a result of the butterfly computation is stored in the memory 11. It should be noted that the width in the butterfly computation may be changed according to actual requirements.

[0093] Please refer to FIG. 12. The data conversion circuit 12 converts the input data into the input matrix according to the target dimension. The input data is the factors of the polynomial and is in vector form. The input data is, for example, data related to fully homomorphic encrypted ciphertext or plaintext. In addition, the target dimension is greater than one dimension (two dimensions in the embodiment). Therefore, the data conversion circuit 12 vectorizes the factors of the polynomial into the input data, and converts the input data into the two-dimensional input matrix. At this time, the column number or the row number of the input matrix may be adjusted according to actual requirements.

[0094] In response to a non-pipelined computation, the butterfly computation is performed through the stage conversion circuit 13.sub.4-1 of one sub-stage, and the output factor of the butterfly computation is stored in the memory 11 through the memory circuit 13.sub.4-2.

[0095] FIG. 14A is a schematic diagram illustrating forward fast number theoretic transform according to an embodiment of the disclosure, and FIG. 14B is a schematic diagram illustrating non-pipelined fast number theoretic transform according to an embodiment of the disclosure. Please refer to FIG. 14B. The data conversion circuit converts 1×16 input data into a 4×4 input matrix, such as

[00020][

15	11	07	03
14	10	06	02
13	09	05	01
12	08	04	00

].

However, the column number or the row number of the input matrix may still be changed according to actual requirements. The “storage” processing in the drawings is, for example, the sub-computation shown in FIG. 13, that is, the butterfly computation is performed and a computation result is stored. The output matrix is

[00021][

18	36	08	67
13	51	72	32
08	67	55	22
27	60	49	08

]

(corresponding to a result of forward fast number theoretic transform shown in FIG. 14A, and the difference is only in the positions of the matrix).

[0096] In addition to two-dimensional, three-dimensional, and four-dimensional multi-dimensional fast number theoretic transform above, higher-dimensional multi-dimensional fast number theoretic transform may also be extended to based on Equation (2). In some application scenarios, higher-dimensional hardware architectures have deeper pipelined depths and longer computing delay, but smaller data throughput for the same number of times. However, higher-dimensional hardware architectures require fewer multipliers and require smaller amounts of data of twiddle factors to be stored. In other words, lower-dimensional hardware architectures have shallower pipelined depths and shorter computing delay, but greater data throughput for the same number of times. However, lower-dimensional hardware architectures require more multipliers and require greater amounts of data of twiddle factors to be stored.

[0097] For example, Table (1) shows the correspondence between the number of multipliers, the number of adders, the number of block memories, and the delay of the target dimension being respectively two-dimensional, three-dimensional, and four-dimensional under a condition that an experimental target is $N=65536$.

TABLE-US-00001 TABLE 1 Number of Number of Number of $N = 65536$ multipliers adders block memories Delay Two-dimensional 2560 4096 64 274 (256×256) Three-dimensional 432 608 240 8337 ($32 \times 32 \times 65$) Four-dimensional 256 256 168 12388 ($16 \times 16 \times 16 \times 16$)

In terms of multiplier resource usage, the architecture of the target dimension being three-dimensional is 1.7 times the architecture of the target dimension being four-dimensional, but the delay is 0.67 times.

[0098] For another example, Table (2) shows the correspondence between the number of multipliers, the number of adders, the number of block memories, and the delay of the target dimension being respectively two-dimensional, three-dimensional, and four-dimensional under a condition that the experimental target is $N=16384$.

TABLE-US-00002 TABLE 2 Number of Number of Number of $N = 16384$ multipliers adders block memories Delay Two-dimensional 1152 1792 32 144 (128×128) Three-dimensional 304 608 240 2193 ($32 \times 32 \times 32$) Four-dimensional 256 256 168 3172 ($16 \times 16 \times 16 \times 4$)

In terms of multiplier resource usage, the architecture of the target dimension being three-dimensional is 1.3 times the architecture of the target dimension being four-dimensional, but the delay is 0.69 times.

[0099] In an embodiment, the processing system 10 includes multiple first computing circuits and multiple second computing circuits. The first computing circuits include the computing circuits 13 for multi-dimensional fast number theoretic transform of a first dimension, the second computing circuits include the computing circuits 13 for multi-dimensional fast number theoretic transform of a second dimension, and the first dimension is different from the second dimension. For example, the first computing circuits include the two stage conversion circuits 13.sub.1-1 and 13.sub.1-4, the first twiddle circuit 13.sub.1-2, and the transposition circuit 13.sub.1-3 (that is, the first dimension is two-dimensional) shown in FIG. 3, and the second computing circuits include the two first twiddle circuits 13.sub.2-1 and 13.sub.2-13, the two second twiddle circuits 13.sub.2-2 and 13.sub.2-12, the three stage conversion circuits 13.sub.2-3, 13.sub.2-7, and 13.sub.2-11, the four transposition circuits 13.sub.2-4, 13.sub.2-6, 13.sub.2-8, and 13.sub.2-10, and the two memory transposition circuits 13.sub.2-5 and 13.sub.2-9 (that is, the second dimension is three-dimensional) as shown in FIG. 7.

[0100] The data conversion circuit 12 may convert the input data into the input matrix that conforms to the first computing circuits or the second

computing circuits according to the target dimension. For example, if the target dimension is two-dimensional, the input data is converted into the input matrix with the column number $\sqrt{\text{square over (N)}}$; and if the target dimension is four-dimensional, the input data is converted into the input matrix with the column number

[00022] $\sqrt[4]{N}$

(d is 4).

[0101] In an embodiment, the data conversion circuit 12 may select one of the first computing circuit and the second computing circuit to perform fast number theoretic transform according to requirements (for example, hardware limitations or delay limitations). For example, if the delay limitation is short, a set of the computing circuits 13 with a smaller target dimension is selected to perform fast number theoretic transform.

[0102] In some embodiments, the processing system 10 may include more computing circuits 13 corresponding to different dimensions. For example, the processing system 10 includes the computing circuits 13 corresponding to two dimensions, three dimensions, and four dimensions.

[0103] FIG. 15 is a flowchart of an encryption method according to an embodiment of the disclosure. Please refer to FIG. 15. The computing circuits 13 are used to perform the fast number theoretic transform calculation, such as two-dimensional fast number theoretic transform shown in FIG. 3, FIG. 4A to FIG. 4C, and FIG. 6A to FIG. 6D, one-dimensional fast number theoretic transform shown in FIG. 5A to FIG. 5C, three-dimensional fast number theoretic transform shown in FIG. 7, FIG. 8A to FIG. 8B, and FIG. 9A to FIG. 9F, four-dimensional fast number theoretic transform shown in FIG. 10 and FIG. 11A to FIG. 11F, or non-pipelined fast number theoretic transform shown in FIG. 12, FIG. 13, and FIG. 14A and FIG. 14B, on the polynomial (Step S1501). The computing circuits 13 perform the matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation (Step S1502). In an embodiment, the elements of the input matrix are the factors of the polynomial used for encryption, such as the factors of the polynomial related to ciphertext or plaintext used for fully homomorphic encryption. In an embodiment, the elements of the input matrix are respectively the same as the elements in the input data (one-to-one element correspondence). The input data is also the factors of the same polynomial and is in vector form. In an embodiment, the fast number theoretic transform is calculated as multi-dimensional fast number theoretic transform of the target dimension. The target dimension may be determined. The target dimension is greater than one dimension. For example, the data conversion circuit 12 determines whether the target dimension is two-dimensional, three-dimensional, or four-dimensional. The input data is converted into the input matrix according to the target dimension. For example, in response to the target dimension being three-dimensional, the data conversion circuit 12 converts the vector input data into the two two-dimensional input matrices. Then, multi-dimensional fast number theoretic transform of the target dimension is performed on the input matrix through the computing circuits 13. Multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a multi-stage computation architecture of the matrix multiplication calculation through multiple sub-computations, one computing circuit 13 performs one sub-computation, and each sub-computation is located at one stage of the multi-stage computation architecture.

[0104] The implementation details of each step in FIG. 15 have been described in detail in the foregoing embodiments and implementations, and will not be described again here.

[0105] In some application scenarios, in addition to a fully homomorphic encryption system, multi-dimensional fast number theoretic transform of the embodiments of the disclosure is also suitable for various lattice cryptography and various applications related to polynomial calculations.

[0106] In some application scenarios, a field programmable gate array (FPGAs) has powerful parallel computing ability and can handle encryption and computing requirements of multiple data blocks at the same time. Implementing the multi-stage hardware architecture of the embodiments of the disclosure through the FPGA may make full use of the parallel computing ability of the FPGA, thereby implementing high-efficiency data processing. However, according to different design requirements, the processing system 10 is not limited to the implementation form of the FPGA.

[0107] In some application scenarios, fully homomorphic encryption can ensure that data during a computation process remains encrypted, thereby ensuring the security of the data. The embodiments of the disclosure can strengthen the security protection and ensure that the data is not exposed to an unauthorized environment at any time.

[0108] In some application scenarios, in big data scenarios, the amount of data is huge and needs to be processed in a timely manner, which puts higher requirements on the traditional fully homomorphic encryption technology. The embodiments of the disclosure can provide the high-efficiency fast number theoretic transform computation and the parallel computing ability of the processing system 10, thereby meeting the processing requirements for large-scale data.

[0109] In some application scenarios, the multiplication computation related to the polynomial in fully homomorphic encryption may be implemented through the processing system 10 of the embodiments of the disclosure, such as an encryption and/or decryption computation on plaintext or ciphertext.

[0110] In summary, in the processing system and method related to encryption of the embodiments of the disclosure, the sub-computations in multi-dimensional fast number theoretic transform are respectively implemented through the corresponding computing circuits, and the pipelined and non-pipelined architectures are provided. In this way, the computing efficiency of fast number theoretic transform can be improved, and fully homomorphic encryption or other lattice-based encryption algorithms are also suitable.

[0111] Although the disclosure has been disclosed in the above embodiments, the embodiments are not intended to limit the disclosure. Persons skilled in the art may make some changes and modifications without departing from the spirit and scope of the disclosure. Therefore, the protection scope of the disclosure shall be defined by the appended claims.

Claims

1. A processing system related to encryption, comprising: a plurality of computing circuits, performing a fast number theoretic transform (NTT) calculation on a polynomial, comprising: performing a matrix multiplication calculation on the polynomial through the fast number theoretic transform calculation.

2. The processing system related to encryption according to claim 1, further comprising: at least one memory, coupled to the computing circuits and configured to store data; a data conversion circuit, coupled to the at least one memory and the computing circuits, and configured to convert input data into an input matrix according to a target dimension, wherein the input data is factors of the polynomial and is in vector form, the target dimension is greater than one dimension, the fast number theoretic transform calculation is multi-dimensional fast number theoretic transform, and the computing circuits are further configured to: perform the multi-dimensional fast number theoretic transform of the target dimension on the input matrix, wherein the multi-dimensional fast number theoretic transform converts matrix-vector multiplication into a multi-stage computation architecture of the matrix multiplication calculation through a plurality of sub-calculations, one computing circuit performs one sub-calculation, and each of the sub-calculations is located at one stage of the multi-stage computation architecture.

3. The processing system related to encryption according to claim 2, wherein: in response to the target dimension being two-dimensional, a mathematical expression of the multi-dimensional fast number theoretic transform is:

$$X_{N_2 k_1 + k_2} = \sum_{n_1=0}^{N_1-1} \left(\sum_{n_2=0}^{N_2-1} x_{N_1 n_2 + n_1} \left(2^{k_2+1} \right)^{N_1 n_2} \right) \left(2^{k_1+1} \right)^{N_2 n_1} \quad \text{where } X_{\text{sub.N.sub.2.sub.k.sub.1.sub.+k.sub.2}} \text{ is an output}$$

factor of the (N.sub.2k.sub.1+k.sub.2)-th multi-dimensional fast number theoretic transform, x.sub.N.sub.1.sub.n.sub.2.sub.+n.sub.1 is the (N.sub.1n.sub.2+n.sub.1)-th input matrix, ω is a twiddle factor, N.sub.1.Math.N.sub.2 is N.sub.1.N.sub.2, and N is a degree of the polynomial.

4. The processing system related to encryption according to claim 3, wherein the computing circuits comprise at least one stage conversion circuit, a first twiddle circuit, and a transposition circuit, a sub-computation of the at least one stage conversion circuit is a butterfly computation, a sub-

16. The method related to encryption according to claim 15, wherein the computing circuits comprise at least one first twiddle circuit, at least one

second twiddle circuit, at least one stage conversion circuit, a transposition circuit, and at least one memory transposition circuit, the method further comprising: performing a multiplication computation of the twiddle factor through the at least one first twiddle circuit and the at least one second twiddle circuit; performing a butterfly computation through the at least one stage conversion circuit; performing factor position swapping through the transposition circuit; and performing memory address change through the memory transposition circuit.

17. The method related to encryption according to claim 12, wherein performing the multi-dimensional fast number theoretic transform of the target dimension on the input matrix through the computing circuits comprises: in response to the target dimension being four-dimensional, determining a mathematical expression of the multi-dimensional fast number theoretic transform as:

$$X_{N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4} = \prod_{n_2=0}^{N_2-1} \left(\prod_{n_3=0}^{N_3-1} \left(\prod_{n_4=0}^{N_4-1} X_{N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4} \right)^{\omega^{N_1 N_2 N_3 n_4}} \right)^{\omega^{2k_4}} \left(\prod_{n_4=0}^{N_4-1} \right)^{N_1 N_2 N_3 n_4} \left(\prod_{n_3=0}^{N_3-1} \right)^{2N_4 k_3} \left(\prod_{n_2=0}^{N_2-1} \right)^{N_1 N_2 n_3 + N_1 n_2 + n_1} \left(\prod_{n_1=0}^{N_1-1} \right)^{2k_2} \omega^{N_4 k_3}$$

where $X_{N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4}$ is an output factor of the $(N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4)$ -th multi-dimensional fast number theoretic transform, $x_{N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4}$ is the $(N_4 N_3 N_2 k_1 + N_4 N_3 k_2 + N_4 k_3 + k_4)$ -th input matrix, ω is a twiddle factor, N_1, N_2, N_3, N_4 is N , and N is a degree of the polynomial.

18. The method related to encryption according to claim 17, wherein the computing circuits comprise at least one first twiddle circuit, at least one stage conversion circuit, at least one transposition circuit, and at least one memory transposition circuit, the method comprising: performing a multiplication computation of the twiddle factor through the at least one first twiddle circuit; performing a butterfly computation through the at least one stage conversion circuit; performing factor position swapping through the at least one transposition circuit; and performing memory address change through the at least one memory transposition circuit.

19. The method related to encryption according to claim 12, wherein: a mathematical expression of the multi-dimensional fast number theoretic

transform is: $X_{k_1, k_2, \dots, k_d} = \prod_{n_1=0}^{N_1-1} \left(\prod_{n_2=0}^{N_2-1} \left(\prod_{n_3=0}^{N_3-1} \left(\prod_{n_d=0}^{N_d-1} X_{k_1, k_2, \dots, k_d} \right)^{\omega^{N_1 N_2 N_3 n_d}} \right)^{\omega^{2k_d}} \right)^{\omega^{2N_4 k_3}} \left(\prod_{n_d=0}^{N_d-1} \right)^{N_1 N_2 N_3 n_d} \left(\prod_{n_3=0}^{N_3-1} \right)^{2N_4 k_3} \left(\prod_{n_2=0}^{N_2-1} \right)^{N_1 N_2 n_3 + N_1 n_2 + n_1} \left(\prod_{n_1=0}^{N_1-1} \right)^{2k_2} \omega^{N_4 k_3}$ where $X_{k_1, k_2, \dots, k_d}$ is an output factor of $(k_1, k_2, \dots, k_d)$ -th multi-dimensional fast number theoretic transform, $x_{k_1, k_2, \dots, k_d}$ is the $(k_1, k_2, \dots, k_d)$ -th input matrix, ω is a twiddle factor, N_1, N_2, N_3, N_d is N , and N is a degree of the polynomial.

20. The method related to encryption according to claim 12, wherein the computing circuits comprise a stage conversion circuit and a memory circuit, the method further comprising: performing a butterfly computation through the stage conversion circuit; and storing data to at least one memory through the memory circuit.